

Challenge Write-Up Template

Instructions: Copy this template for every challenge you submit. Fill in every section — a write-up that only shows the final flag without explanation will not receive full credit. Delete these italicized instruction lines before submitting.

Challenge Information

Student Name	Facon, Janna Mae I.
Section / Class	BSIT-3G
Challenge Name	Bytemancy 1
Category	General Skills
Points / Difficulty	Easy
Date Solved	8/12/26
Flag Format Given	PicoCTF{...}

1. Challenge Description

Restate the challenge in your own words: what were you given, and what were you asked to find or do? Do not simply copy-paste the original prompt.

The ByteMancy 1 challenge required me to connect to a remote server using Netcat and send a specific ASCII character a certain number of times. The challenge asked for ASCII decimal value **101** to be sent **1,751 times**, side-by-side without spaces. The goal was to determine which character represents ASCII decimal 101, generate the required number of characters efficiently, send them to the server, and obtain the flag.

2. Initial Analysis / Reconnaissance

What did you check first? What did file, strings, a webpage view-source, or another first look tell you? What category did you decide this was, and why?

The first thing I did was read the challenge description and noticed that it was categorized as a **General Skills** challenge. The challenge was part of the ByteMancy series and mentioned that the program's source code could be downloaded. I then launched the challenge instance and connected to the server using Netcat. The server instructed me to send **ASCII decimal 101 1,751 times, side-by-side, with no spaces**. I recognized that ASCII decimal **101** represents the lowercase letter **e**, so I needed to send the letter **e** exactly 1,751 times. Because manually typing the character that many times would be inefficient and could cause mistakes, I decided to use a Python one-liner to generate the required characters automatically.

3. Tools and Commands Used

List every tool/command you used, in the order you used them, and what each one told you.

Tool / Command	What it was used for	Result / Output
Challenge description	To understand the requirements and identify what the server was asking for.	The challenge required sending ASCII decimal 101 a specific number of times.
nc foggy-cliff.picoctf.net 50323	To connect to the remote ByteMancy 1 server.	The server asked for ASCII decimal 101, 1,751 times, side-by-side with no spaces.
ASCII conversion	To determine which character corresponds to decimal 101.	ASCII decimal 101 corresponds to lowercase e.
python -c 'print("\x65" * 1752)' nc foggy-cliff.picoctf.net 50323	To automatically generate and send repeated e characters through Netcat.	This sent 1,752 e characters, which was one character too many. The server responded that the answer was incorrect.
python -c 'print("\x65" * 1751)' nc foggy-cliff.picoctf.net 50323	To correct the previous attempt by generating exactly 1,751 e characters.	The server accepted the correct input and returned the flag: picoCTF{h0w_m4ny_e's???_b6277f00}.

4. Solution Walkthrough

Explain your reasoning step by step — not just what you typed, but why you tried it and what you concluded from the result. Include screenshots, code snippets, or command output where useful. A reader who has not seen the challenge should be able to follow your logic and arrive at the same answer.

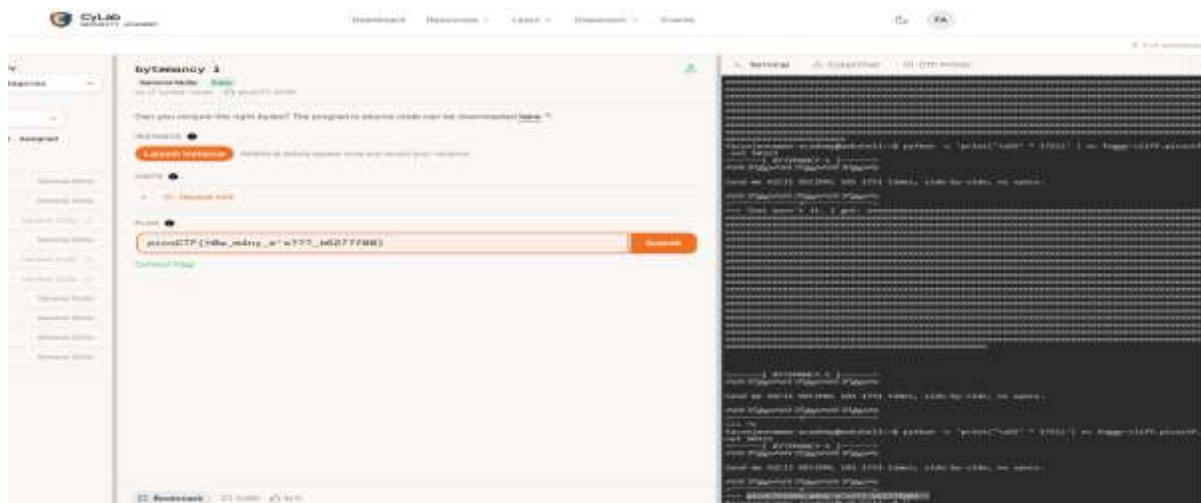
Step 1

The first step I did was read the challenge description to understand what I needed to accomplish. I identified that ByteMancy 1 was a **General Skills** challenge involving ASCII values and communication with a remote server. After launching the instance, I connected to the server using Netcat. The server instructed me to send **ASCII decimal 101 exactly 1,751 times**, side-by-side with no spaces.



Step 2

I determined that ASCII decimal **101** represents the lowercase letter **e**. Since I needed to send the character 1,751 times, manually typing e repeatedly would be inefficient and could easily result in the wrong number of characters. I therefore used a Python one-liner to generate the characters automatically: `python -c 'print("\x65" * 1751)' | nc foggy-cliff.picoctf.net 50323` The `\x65` represents the hexadecimal ASCII value for e, and multiplying it by 1751 generates the required number of characters. I also piped the generated output directly into Netcat so that it would be sent to the challenge server.



Step 3

After seeing that the first attempt was incorrect, I stopped the connection and corrected the number from **1752 to 1751**. I then ran: `python -c 'print("\x65" * 1751)' | nc foggy-cliff.picoctf.net 50323` This time, the command generated exactly 1,751 e characters and sent them to the server. The server accepted the input and returned the flag: `picoCTF{h0w_m4ny_e's???_b6277f00}` I then copied the flag and entered it into the **FLAG** field on the ByteMancy 1 challenge page.



5. Flag

```
picoCTF{h0w_m4ny_e's???_b6277f00}
```

6. Key Concept / What I Learned

In 2-4 sentences, name the underlying security concept or vulnerability class this challenge taught (e.g. "weak RSA exponent," "reflected XSS," "stack buffer overflow"). This is the part of the write-up most worth remembering after the flag is submitted.

*This challenge taught me about **ASCII encoding and automated byte generation**. I learned that ASCII decimal 101 represents the character *e*, and that Python can be used to generate a specific number of characters efficiently. I also learned the importance of sending the **exact required number of bytes**, because sending even one extra character caused the server to reject the answer.*

7. References

List any external resource you consulted beyond course materials (a tool's documentation, a technique explainer, a similar public write-up). It is normal and expected to use references — just cite them.

<https://youtu.be/jifzhZkcLx8?si=O6Z6Rhqa03a6c55Z>
